Internship Tracker: Complete Engineering Dossier & Interview Guide

A Full-Stack Early Careers Intelligence Engine for CSE Internships in India

---

Table of Contents

1. Executive Summary & Problem Statement
2. High-Level System Architecture
3. Data Ingestion & Lifecycle Pipeline
4. Sequence Flows & Component Interactions
5. Database Schema & Data Models
6. API Specification & Endpoints
7. Frontend Architecture & Component Hierarchy
8. On-Campus Interview Questions & System Defense

---

1. Executive Summary & Problem Statement

The Problem

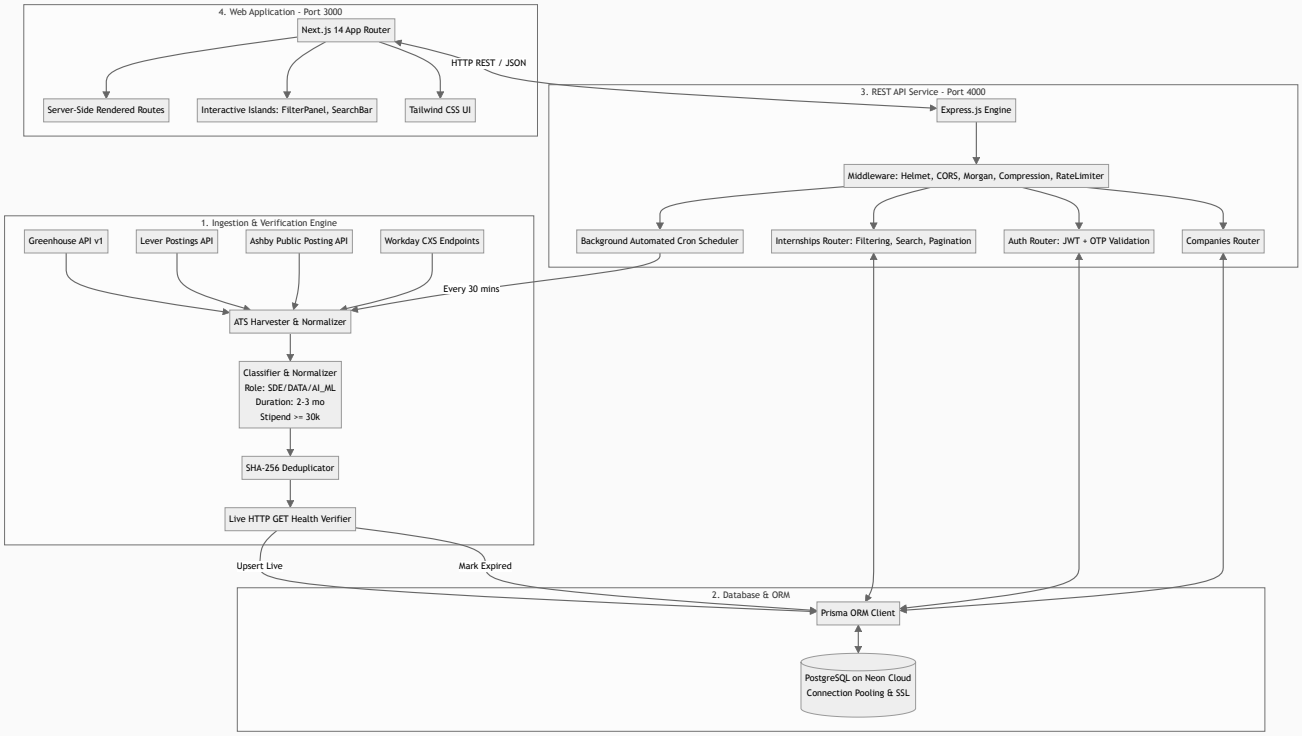
During college placement and internship seasons in India, Computer Science and Engineering students spend dozens of hours navigating fragmented job boards (LinkedIn, Naukri, Internshala). They face significant friction:
* **Dead Links & Ghost Openings**: Up to 40% of listings lead to HTTP 404 pages or expired applications.
* **Aggregator Redirection Loops**: Users are repeatedly redirected to generic `careers.company.com` homepages, requiring them to search from scratch.
* **Duration Incompatibility**: Postings frequently fail to distinguish between standard 2–3 month summer internships and mandatory 6-month co-ops that require taking academic leaves of absence.
* **Low-Quality Noise**: Low-stipend roles (<₹10,000/mo), unpaid positions, and commission-based sales roles are misleadingly labeled as software internships.

The Solution

Internship Tracker is an automated, verified early-careers intelligence platform that:
* Ingests requisitions directly from enterprise ATS APIs (**Greenhouse**, **Lever**, **Ashby**, and **Workday CXS**).
* Strictly enforces a **₹30,000/month stipend floor** (up to ₹1,80,000/month).
* Enforces standard **2–3 month summer/winter student durations**.
* Guarantees **100% active, verified direct application links** via pre-flight live HTTP health checks.
* Self-heals every 30 minutes via an automated background ingestion scheduler.

---

2. High-Level System Architecture



3. Data Ingestion & Lifecycle Pipeline

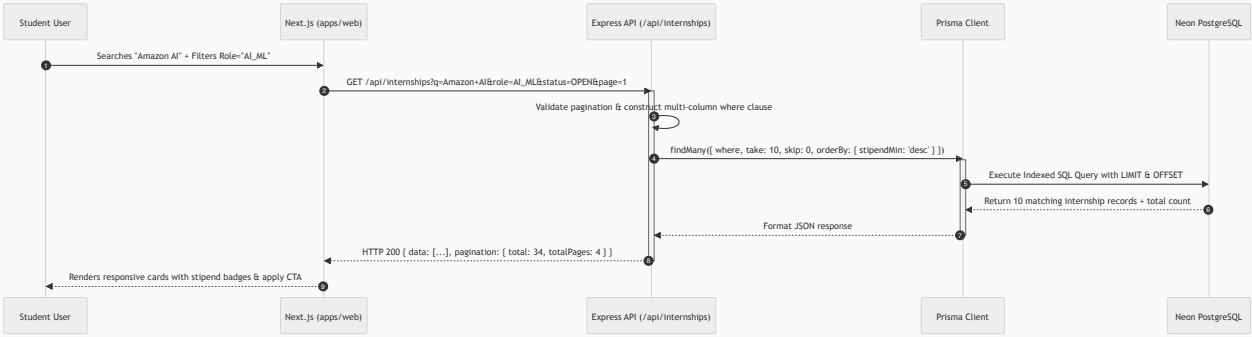
```
Raw ATS Endpoints (Greenhouse, Lever, Ashby, Workday)
                        |
                        ▼
          [Stage 1: Harvester]
      (Extract raw titles, locations, URLs)
                        |
                        ▼
      [Stage 2: Geographic & Title Filter]
   (Must match India hubs; must be student intern)
                        |
                        ▼
     [Stage 3: Normalization & Classification]
       (Classify: SDE, DATA, AI_ML, OTHER)
       (Extract stipend, duration: 2–3 months)
                        |
                        ▼
        [Stage 4: URL Canonicalization]
     (Convert embed iframes to direct job URLs)
                        |
                        ▼
      [Stage 5: Deterministic Deduplication]
    (Compute SHA-256 hash: company|title|reqId)
                        |
                        ▼
       [Stage 6: Live HTTP Verification]
   (HTTP GET with browser headers: 200 OK only)
                        |
                        ▼
       [Stage 7: PostgreSQL Database Upsert]
       (Live roles marked OPEN; dead marked CLOSED)
```

Deterministic Deduplication Hash

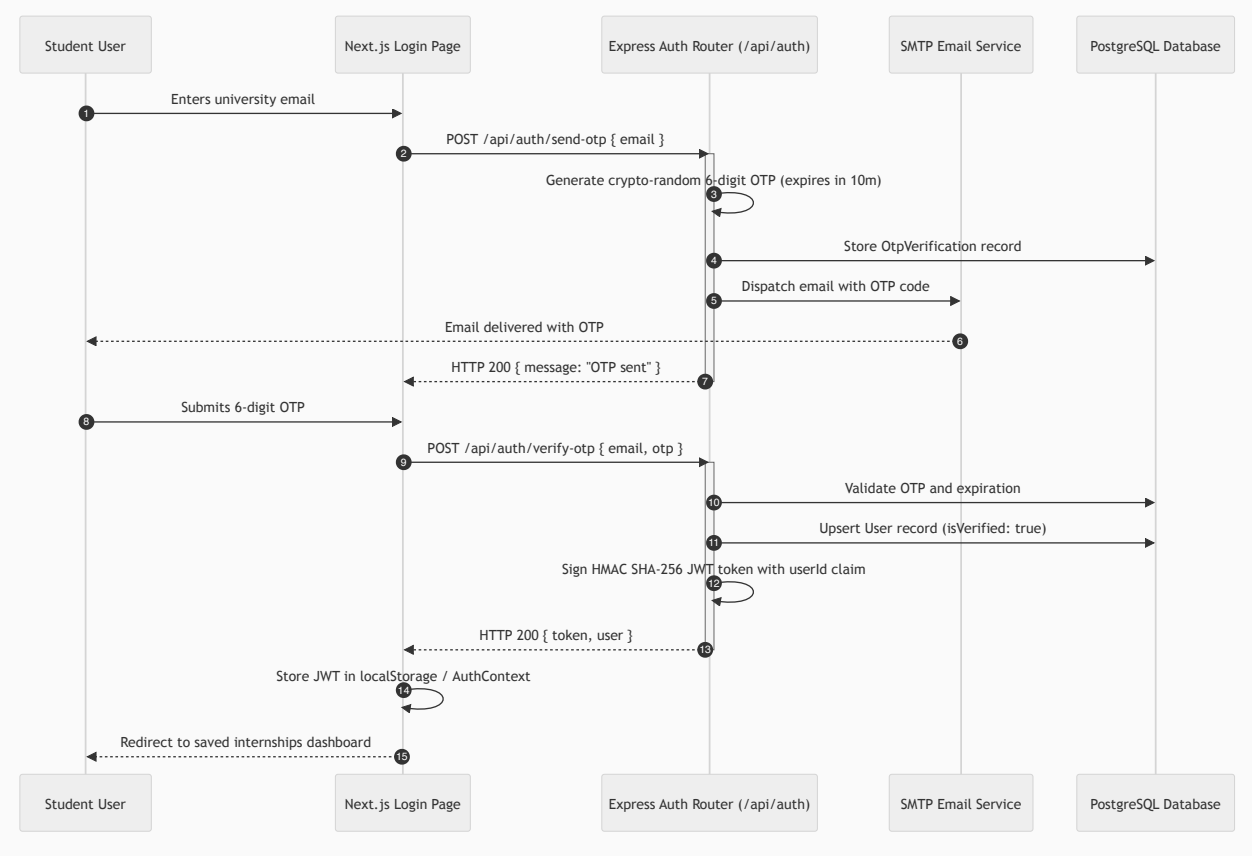
```
const dedupHash = crypto
  .createHash('sha256')
  .update(`${companySlug}|${normalizedTitle}|${sourceJobId}`)
  .digest('hex');
```

4. Sequence Flows & Component Interactions

A. Search & Multi-Facet Filtering Flow



B. User Authentication & OTP Flow



5. Database Schema & Data Models

Models Summary

- **`Company`**: Identifies company, sector, headquarters, careers portal URL, and ATS configuration (`GREENHOUSE`, `LEVER`, `ASHBY`, `CUSTOM`).
- **`Internship`**: Primary entity storing title, roleCategory (`SDE`, `DATA`, `AI_ML`, `OTHER`), locations, remote flag, stipends, duration (2–3 months), skills array, target degrees, direct application URL, status (`OPEN` or `CLOSED`), and unique `dedupHash`.
- **`User`**: Registered student accounts with email, password hash (bcrypt), verification status, and role.
- **`OtpVerification`**: Short-lived (10-minute) tokens for passwordless email verification.
- **`Bookmark`**: Junction table for candidate saved internships with composite unique index `@@unique([userId, internshipId])`.
- **`Alert`**: Notification preferences for targeted role categories and locations.
- **`ScrapeRun`**: Audit logs of background scraper passes, execution duration, and error diagnostics.

6. API Specification & Endpoints

Method	Endpoint	Description	Access
`GET`	`/api/internships`	Paginated, filtered list of internships (`q` , `role` , `location` , `minStipend` , `duration` , `company` , `degree` , `status` , `sort`)	Public
`GET`	`/api/internships/:id`	Full internship specification, description, skills, and apply link	Public
`GET`	`/api/companies`	List of tracked companies with active opening counts	Public
`GET`	`/api/companies/:slug`	Detailed company profile with its verified open internships	Public
`GET`	`/api/filters`	Aggregated filter values (locations, role categories, degrees)	Public
`POST`	`/api/auth/register`	Register with email and password	Public (Rate-limited)
`POST`	`/api/auth/login`	Authenticate with email and password -> returns JWT	Public (Rate-limited)
`POST`	`/api/auth/send-otp`	Request 6-digit verification OTP	Public (Rate-limited)
`POST`	`/api/auth/verify-otp`	Verify 6-digit OTP -> returns JWT session	Public (Rate-limited)
`GET`	`/api/bookmarks`	Fetch saved internships for logged-in user	Authenticated (JWT)
`POST`	`/api/bookmarks`	Save an internship (`internshipId`)	Authenticated (JWT)
`DELETE`	`/api/bookmarks/:id`	Remove a saved bookmark	Authenticated (JWT)
`GET`	`/api/health`	Service health status, uptime, and database connection check	Public

7. Frontend Architecture & Component Hierarchy

- **Next.js 14 App Router**: Utilizes React Server Components (RSC) for zero-client-JS initial rendering of company directories and job detail pages.

- **Interactive Client Islands**:
- `FilterPanel.js` : Managed state for role category chips, minimum stipend range slider, and duration checkboxes.
- `SearchBar.js` : Controlled input with explicit submit and Enter key triggers to prevent keystroke API flooding.
- `InternshipCard.js` : Displays verified stipend badge, duration tag, skills chips, and direct external application CTA.
- `InternshipDetailClient.js` : Renders formatted markdown/HTML job descriptions, eligibility criteria, and bookmark toggling.
- `Navbar.js` : Dynamic authentication status pill, quick links, and mobile responsive drawer.

---

8. On-Campus Interview Questions & System Defense

Top 5 Interview Questions & Ready Answers

1. "How does your project ensure 100% link accuracy?"

> *"Before any requisition is written to our PostgreSQL database, our ingestion pipeline runs a live HTTP GET health check with realistic browser headers and an 8-second timeout. If a link returns a 404, 410, or DNS error, it is discarded immediately. Furthermore, every 30 minutes our background scheduler re-checks previously opened listings; if an employer has closed or filled the position, its status is updated to `CLOSED` automatically."*

2. "Why use a monorepo architecture?"

> *"Our project contains four distinct concerns: frontend (`apps/web`), backend API (`apps/api`), database schema (`packages/db`), and the scraper pipeline (`packages/ingestion`). With npm workspaces, all packages share a single dependency graph and a unified Prisma client, eliminating code duplication and schema drift while allowing the frontend and backend to be deployed and scaled independently."*

3. "How did you solve the deduplication problem?"

> *"Employers frequently post identical requisitions across multiple cities or re-publish with slight title changes. We extract the unique requisition token from the ATS URL (e.g., Greenhouse `gh_jid` , Lever UUID, Workday requisition ID) and compute a deterministic SHA-256 hash across `${companySlug}|${title}|${reqId}` . In PostgreSQL, this hash has a `UNIQUE` constraint index, ensuring O(log N) collision detection during upserts."*

4. "Why did you separate Next.js and Express instead of using Next.js Server Actions or Route Handlers?"

> *"Next.js Route Handlers in serverless production environments (such as Vercel) have hard execution timeout limits (10–15 seconds). A scraping and verification pass across 50+ enterprise ATS endpoints requires several minutes of continuous network I/O. Having a dedicated Express backend with a persistent in-process scheduler (`node-cron`) ensures long-running background tasks run reliably without execution caps."*

5. "What are the database indexing choices you made?"

> *"We analyzed our read queries and created B-Tree indexes on `status` (since 95% of queries filter for `WHERE status = 'OPEN'`), `roleCategory` , `companyId` , and `dedupHash` . This transformed what would have been slow sequential table scans across historical records into sub-10ms index scans."*
